

Phase 1: Data Ingestion Pipeline — Complete Learning Guide

Table of Contents

1. The Big Picture (What & Why)
 2. Project Setup (How to Start)
 3. File 1: create_sample_data.py (The Test Data Factory)
 4. File 2: schema.py (The Data Bouncer)
 5. File 3: m5_preprocessor.py (The Translator)
 6. File 4: pipeline.py (The Orchestra Conductor)
 7. File 5: test_ingestion.py (The Safety Net)
 8. How to Execute Everything
 9. Metrics & What They Mean
 10. Interview Preparation
-

1. The Big Picture

THE BUSINESS PROBLEM

You work at a semiconductor company (think: companies like Applied Materials, Intel, TSMC). Your warehouse stores thousands of different spare parts — chips, cables, sensors, motors.

Every month, factories call: "I need 500 units of Part X by Friday!"

Three things can go wrong:

1. **TOO FEW parts in stock** → Factory machines break down → Production stops → Company loses millions per hour
2. **TOO MANY parts in stock** → Warehouse costs \$\$\$\$ → Parts become obsolete → Company wastes money
3. **WRONG predictions** → You ordered 10,000 of Part A but needed Part B → Both problems above happen at once

Current state (without this project): A supply chain manager opens Excel, stares at 46 million rows, manually tries to spot patterns, makes educated guesses. This takes hours and is often wrong.

Future state (with this project): The manager types: "Which parts are at risk of stockout next quarter?" The AI searches the data, runs ML forecasts, and answers in plain English with citations — in 2 seconds.

THE TECHNICAL APPROACH

We build this in 4 phases:

Phase 1 (YOU ARE HERE): Clean data pipeline

→ Raw messy data goes in, clean validated data comes out

Phase 2: ML forecasting engine

→ Clean data goes in, demand predictions come out

Phase 3: GenAI/RAG layer

→ English questions go in, data-grounded answers come out

Phase 4: Deployment

→ Package everything into an API + dashboard anyone can use

WHY PHASE 1 MATTERS MOST

There's a saying in data science: "Garbage in, garbage out."

If you feed your ML model data where:

- demand_quantity is "hello" instead of a number
- dates are formatted as "Jan 5" in some rows and "2015-01-05" in others
- some part numbers are missing
- negative quantities exist (physically impossible)

...then your model will either crash or silently produce garbage predictions that LOOK correct but aren't. Phase 1 eliminates this entire class of bugs.

2. Project Setup

STEP 1: Create the project folder structure

```
bash
```

```
mkdir -p agentic-supply-chain-orchestrator
cd agentic-supply-chain-orchestrator

# Create all folders
mkdir -p data/raw data/processed
mkdir -p src/ingestion src/forecasting src/genai src/api
mkdir -p tests notebooks

# Create __init__.py files (tells Python these are packages)
touch src/__init__.py
touch src/ingestion/__init__.py
touch src/forecasting/__init__.py
touch src/genai/__init__.py
touch src/api/__init__.py
touch tests/__init__.py
```

TECHNICAL: What is `init.py`? Python needs a file called `init.py` inside a folder to treat that folder as a "package" (importable module). Without it, `from src.ingestion import schema` would fail with "ModuleNotFoundError". The file can be empty — its existence is what matters.

BUSINESS: Why this folder structure? Separation of concerns. Each folder has ONE job:

- `src/ingestion/` → data cleaning (Phase 1)
- `src/forecasting/` → ML models (Phase 2)
- `src/genai/` → AI reasoning (Phase 3)
- `src/api/` → web endpoints (Phase 4)
- `tests/` → automated tests for everything
- `data/raw/` → untouched original data
- `data/processed/` → cleaned, validated data

If a bug appears in forecasting, you look in `src/forecasting/`.

If data quality drops, you look in `src/ingestion/`.

Each team member can work on their folder without stepping on others.

Your final folder structure should look like this:

```
agentic-supply-chain-orchestrator/
├─ data/
│   └─ raw/                               ← Original data goes here
│       └─ create_sample_data.py
│       └─ sales_train_evaluation.csv
```

```

|   |   └─ sell_prices.csv
|   |   └─ calendar.csv
|   └─ processed/           ← Cleaned data ends up here
|       └─ m5_amat_mapped.csv
|       └─ m5_amat_validated.csv
└─ src/
    |   └─ __init__.py
    |   └─ ingestion/
    |       |   └─ __init__.py
    |       |   └─ schema.py           ← Data validation rules
    |       |   └─ m5_preprocessor.py ← M5 → AMAT column mapping
    |       └─ pipeline.py           ← ETL orchestration
    └─ forecasting/                ← Phase 2 (empty for now)
        └─ genai/                  ← Phase 3 (empty for now)
            └─ api/                 ← Phase 4 (empty for now)
└─ tests/
    |   └─ __init__.py
    └─ test_ingestion.py           ← 36 tests for Phase 1
└─ requirements.txt
└─ .env.example
└─ README.md

```

STEP 2: Install dependencies

```

bash

python -m venv venv           # Create virtual environment
source venv/bin/activate      # Activate it (Mac/Linux)
# OR: venv\Scripts\activate   # Activate it (Windows)

pip install -r requirements.txt # Install all libraries

```

TECHNICAL: What is a virtual environment? It's an isolated Python installation. Without it, installing libraries for this project might break other projects on your computer. Think of it like a sandbox — whatever you install inside stays inside.

3. File 1: create_sample_data.py — The Test Data Factory

WHAT THIS FILE DOES

Creates 1,440 rows of fake data that mimics the real M5 dataset (which has 46 million+ rows). This lets you develop and test without

downloading a 3GB dataset from Kaggle.

THE REAL M5 DATASET

The M5 dataset comes from a Kaggle competition sponsored by Walmart.

It contains EVERY item sold across 10 Walmart stores over 5 years.

Three CSV files:

sales_train_evaluation.csv	– What sold, where, when (46M+ rows)
sell_prices.csv	– Weekly prices for each item at each store
calendar.csv	– Date info: day of week, events, SNAP days

WHY WE DON'T USE REAL SEMICONDUCTOR DATA

Real supply chain data is proprietary (confidential). No semiconductor company would give you their parts inventory data for a portfolio project.

The M5 dataset is public, has REAL patterns (seasonality, trends, spikes), and is the academic gold standard for forecasting research.

CODE WALKTHROUGH

```
python

import numpy as np

np.random.seed(42)
```

TECHNICAL: `random.seed(42)` makes "random" numbers reproducible. Every time you run this script, you get the EXACT same "random" data. Without it, every run produces different data, making debugging impossible. The number 42 is arbitrary (it's a Douglas Adams reference).

BUSINESS: Reproducibility is critical in production. If a bug appears, you need to reproduce it exactly to fix it.

```
python

ITEMS = [
    "HOBBIES_1_001", "HOBBIES_1_008", ... # → NPI demand
    "HOUSEHOLD_1_012", ...               # → Standard demand
    "FOODS_1_001", ...                   # → Service campaign
    "FOODS_3_042", ...                   # → Reliability
]
```

TECHNICAL: These are real M5 item IDs. Each one has a naming pattern:

`CATEGORY_DEPARTMENT_ITEMID`. We include items from every department so our sample data covers all 4 demand types after mapping.

```
python
```

```
base = np.random.poisson(5) # Spikey, unpredictable (for HOBBIES)
base = np.random.poisson(12) # Steady, predictable (for HOUSEHOLD)
```

TECHNICAL: `np.random.poisson(5)` generates random count data with an average of 5. Poisson distribution is used because:

1. Sales are COUNT data (you sell 0, 1, 2, 3... items, not 2.7 items)
2. Poisson is the standard distribution for "number of events per period"
3. It naturally produces occasional spikes (rare high values)
4. It can't produce negative numbers (you can't sell -3 items)

BUSINESS: Different departments use different Poisson averages because different types of parts have different demand levels:

- HOBBIES (NPI) → low average (5), high variance = spikey demand
- HOUSEHOLD (Standard) → higher average (12) = steady demand

```
python
```

```
if date.weekday() >= 5: # Saturday=5, Sunday=6
    base = int(base * 1.3) # 30% more on weekends
```

TECHNICAL: This adds seasonality — a recurring pattern. Real retail data shows higher weekend sales. Our ML models need to learn this pattern.

BUSINESS: Seasonality exists in semiconductor supply chains too. Q4 demand spikes because consumer electronics companies build inventory for holiday sales. ARIMA (Phase 2) is specifically designed to capture these patterns.

HOW TO RUN

```
bash
```

```
cd agentic-supply-chain-orchestrator
python data/raw/create_sample_data.py
```

Expected output:

WHAT TO VERIFY

After running, check that three CSV files exist:

```
bash

ls -la data/raw/*.csv
# Should show:
#   sales_train_evaluation.csv  (~80KB)
#   sell_prices.csv            (~8KB)
#   calendar.csv               (~2KB)
```

Open one and spot-check:

```
bash

head -5 data/raw/sales_train_evaluation.csv
# Should show columns: item_id, dept_id, cat_id, store_id, ...
```

4. File 2: schema.py — The Data Bouncer

WHAT THIS FILE DOES

Defines RULES for what valid data looks like, then checks every single record against those rules. Bad records get rejected with clear error messages.

THE BUSINESS NEED

In a real supply chain system, data arrives from many sources:

- Manual entry by warehouse staff (typos happen)
- Automated sensors (sensors malfunction)
- ERP systems (systems have bugs)
- Third-party suppliers (different formats)

Without validation, bad data corrupts everything downstream.

A single record with `demand_quantity = -500` could make your ML model predict negative demand, which makes your ordering system try to RETURN inventory that was never ordered.

THE CONCEPT: WHAT IS PYDANTIC?

Pydantic is a Python library for data validation. You define a class with type hints, and Pydantic automatically:

1. Checks that every field has the right type
2. Converts compatible types (e.g., string "42" → integer 42)
3. Rejects invalid data with clear error messages
4. Generates API documentation automatically

In our project, we built Pydantic-style validation BY HAND so you understand what happens under the hood. In production, always use the real Pydantic library — it's faster and handles edge cases.

CODE WALKTHROUGH — PART 1: VALID VALUES

```
python
```

```
VALID_DEMAND_TYPES = {"NPI", "standard", "service_campaign", "reliability"}  
VALID_REGIONS = {"North America", "Asia Pacific", "Europe"}
```

TECHNICAL: These are Python sets (curly braces). Sets are used instead of lists because checking "is X in this set?" is $O(1)$ — instant, no matter how big the set. With a list, Python checks every element one by one ($O(n)$).

BUSINESS: These represent real-world constraints:

- Only 4 types of demand exist in our supply chain model
- Only 3 geographic regions have AMAT facilities
- Any value outside these sets indicates corrupted data

CODE WALKTHROUGH — PART 2: THE RECORD CLASS

```
python
```



```
class SupplyChainRecord:
    def __init__(self, part_number, demand_type, region,
                  demand_quantity, transaction_date, unit_cost,
                  part_description, npi_signal=None):
```

TECHNICAL: `__init__` is Python's constructor — it runs when you create a new `SupplyChainRecord`. Every parameter corresponds to one column in our processed CSV.

`npi_signal=None` means this parameter is OPTIONAL. Not every record has an NPI event.
`None` means "no event today."

BUSINESS: Each field maps to a real supply chain concept:

<code>part_number</code>	→ What part is this? (e.g., "CHIP_X7_200nm")
<code>demand_type</code>	→ Why is it needed? (NPI launch? routine? recall?)
<code>region</code>	→ Where is it needed? (which geographic market?)
<code>demand_quantity</code>	→ How many units? (the number we'll predict in Phase 2)
<code>transaction_date</code>	→ When was it needed? (for time-series analysis)
<code>unit_cost</code>	→ How much per unit? (affects ordering decisions)
<code>part_description</code>	→ What category? (for grouping in reports)
<code>npi_signal</code>	→ Was there a product launch? (explains demand spikes)

CODE WALKTHROUGH — PART 3: VALIDATION RULES

```
python

def _validate(self):
    errors = []

    # Check 1: part_number must be a non-empty string
    if not isinstance(self.part_number, str) or len(self.part_number.strip()) == 0:
        errors.append("part_number must be a non-empty string")
```

TECHNICAL: `isinstance(x, str)` checks if `x` is a string type. `.strip()` removes whitespace from both ends — so `" "` (just spaces) is treated as empty. We ACCUMULATE errors in a list instead of raising immediately because...

```
python
```

```
# At the end:
if errors:
    raise ValueError(
        f"Validation failed for record:\n" +
        "\n".join(f" - {e}" for e in errors)
    )
```

TECHNICAL: Error accumulation vs fail-fast:

- **Fail-fast:** Stop at the FIRST error. "part_number is empty." Done. You fix it, run again, get "demand_type is invalid." Fix that, run again, get "date is wrong." Three rounds of debugging for one record.
- **Error accumulation:** Collect ALL errors. "part_number is empty AND demand_type is invalid AND date is wrong." One round of debugging.

We use error accumulation because in production, you might have 1000 bad records, each with multiple issues. Fail-fast would require 1000 separate runs.

python

```
# Check: demand_quantity must be integer >= 0
if isinstance(self.demand_quantity, (int,)) and self.demand_quantity < 0:
    errors.append(f"demand_quantity cannot be negative (got {self.demand_quantity})")
```

BUSINESS: Why can't demand be negative?

- Demand means "how many units were requested." You can't request -50 parts.
- Zero IS valid — it means "nobody needed this part today."
- Negative values would mean "we un-requested parts" which isn't real.
- If negative values sneak into training data, the ML model might predict negative future demand, causing the ordering system to malfunction.

python

```
# Check: transaction_date must be valid YYYY-MM-DD
try:
    datetime.strptime(self.transaction_date, "%Y-%m-%d")
except (ValueError, TypeError):
    errors.append(f"transaction_date '{self.transaction_date}' is not valid.")
```

TECHNICAL: `strptime` parses a string into a date object. If the string doesn't match the format, it raises `ValueError`. This catches:

- "2015-13-45" (month 13 doesn't exist)
- "January 5, 2015" (wrong format)
- "hello" (not a date at all)
- `None` (not even a string)

BUSINESS: Time-series models (ARIMA in Phase 2) need dates in a consistent format to calculate intervals. If one record says "Jan 5" and another says "2015-01-05", the model can't compute "how many days between these records?"

CODE WALKTHROUGH — PART 4: BATCH VALIDATOR

python

```
def validate_batch(records: list[dict]) -> tuple[list[SupplyChainRecord], list[dict]]:
    valid_records = []
    errors = []

    for i, record_dict in enumerate(records):
        try:
            record = SupplyChainRecord(**record_dict)
            valid_records.append(record)
        except (ValueError, TypeError) as e:
            errors.append({"row": i, "error": str(e)})

    return valid_records, errors
```

TECHNICAL: `**record_dict` is Python's "dictionary unpacking." If `record_dict = {"part_number": "X", "demand_type": "NPI", ...}`, then `SupplyChainRecord(**record_dict)` is the same as: `SupplyChainRecord(part_number="X", demand_type="NPI", ...)`

The function returns a TUPLE of two lists:

- `valid_records`: all records that passed validation
- `errors`: all records that failed, with row numbers

BUSINESS: In production, you might ingest 100,000 records from a nightly batch job. You want to process all 99,950 good records AND report all 50 bad ones — not stop at the first bad one and lose 99,999 good records.

HOW TO RUN

```
bash

python src/ingestion/schema.py
```

Expected output:

```
Testing schema validation...
```

```
✅ Valid record: SupplyChainRecord(part=HOBBIES_1_001, type=NPI, ...)
```

```
❌ Caught bad record:
```

```
Validation failed for record:
```

- region 'Mars' is not valid. Must be one of: {...}
- demand_quantity cannot be negative (got -5)

```
✅ Schema validation working correctly!
```

5. File 3: m5_preprocessor.py — The Translator

WHAT THIS FILE DOES

Takes the raw M5 (Walmart) CSV columns and renames/maps them to our semiconductor supply chain schema. The DATA PATTERNS stay identical — only the LABELS change.

THE BUSINESS NEED

The M5 dataset speaks "retail Walmart language":

- "items" = things you buy at a store
- "stores" = physical locations
- "departments" = HOBBIES, FOODS, HOUSEHOLD

Our project needs "semiconductor supply chain language":

- "parts" = components for machines
- "regions" = geographic markets
- "demand types" = NPI, standard, service_campaign, reliability

WHY THESE SPECIFIC MAPPINGS

This is the most important design decision in Phase 1. Each mapping was chosen because the SOURCE data's behavior matches the TARGET concept:

HOBBIES → NPI (New Product Introduction)

REAL WORLD: Hobby items (fidget spinners, Pokemon cards) spike suddenly when a trend starts, then fade away.

SUPPLY CHAIN: New chip launches spike suddenly when a product is announced, then stabilize.

BEHAVIOR: Sudden spike → gradual decline. Unpredictable timing.

HOUSEHOLD → Standard

REAL WORLD: Household items (soap, paper towels) sell steadily every week. Very predictable.

SUPPLY CHAIN: Standard replacement parts (screws, cables) have steady demand from factories that always need them.

BEHAVIOR: Flat line with small fluctuations. Very predictable.

FOODS_1, FOODS_2 → Service Campaign

REAL WORLD: Food demand spikes around holidays (Thanksgiving turkey) then crashes back to normal.

SUPPLY CHAIN: Product recalls or warranty campaigns cause a sudden demand spike, then drop when the campaign ends.

BEHAVIOR: Sharp spike → sharp drop. Event-driven.

FOODS_3 → Reliability

REAL WORLD: Some food staples (coffee, bread) show gradual growth as the population and consumption habits change.

SUPPLY CHAIN: Parts that wear out over time show gradually increasing demand as equipment ages across the installed base.

BEHAVIOR: Slow upward trend. Predictable growth rate.

CODE WALKTHROUGH — THE MAPPING DICTIONARIES

python

```
DEPT_TO_DEMAND_TYPE = {  
    "HOBBIES_1": "NPI",  
    "HOBBIES_2": "NPI",  
    "HOUSEHOLD_1": "standard",  
    "HOUSEHOLD_2": "standard",  
    "FOODS_1": "service_campaign",  
    "FOODS_2": "service_campaign",  
    "FOODS_3": "reliability",  
}
```

TECHNICAL: A Python dictionary — the fastest way to do lookups.

`DEPT_TO_DEMAND_TYPE["HOBBIES_1"]` returns `"NPI"` in $O(1)$ time.

python

```
STORE_TO_REGION = {  
    "CA_1": "North America", "CA_2": "North America",  
    "TX_1": "North America", "TX_3": "North America",  
    "WI_1": "Asia Pacific",  
    "WI_3": "Europe",  
}
```

BUSINESS: In the real M5 data, CA = California, TX = Texas, WI = Wisconsin. We map:

- CA + TX (western/southern US) → North America
- WI_1, WI_2 → Asia Pacific
- WI_3 → Europe

The exact mapping is arbitrary since it's fake data. What matters is that we have 3 distinct regions with different demand patterns, which is realistic for a global semiconductor company.

CODE WALKTHROUGH — THE TRANSFORMATION FUNCTION

python

```
def map_columns(sales_df, prices_df):  
    df = sales_df.copy()
```

TECHNICAL: `.copy()` creates a SEPARATE copy of the DataFrame. Without it, changes to `df` would also change `sales_df` (because Python DataFrames are passed by reference, not by value). This protects the original data.

python

```
# Map department → demand type
df["demand_type"] = df["dept_id"].map(DEPT_TO_DEMAND_TYPE).fillna("standard")
```

TECHNICAL: `.map(dictionary)` replaces each value in the column with its dictionary lookup result. If a `dept_id` isn't in the dictionary, it returns `NaN`. `.fillna("standard")` replaces any `NaN` with "standard" as a safe default.

This is a common pandas pattern: `column.map(dict).fillna(default)`.

python

```
# Rename columns
df = df.rename(columns={
    "item_id":      "part_number",
    "sales":        "demand_quantity",
    "date":         "transaction_date",
    "sell_price":   "unit_cost",
    "cat_id":       "part_description",
    "event_name_1": "npi_signal",
})
```

TECHNICAL: `.rename(columns=dict)` changes column names without touching the data. `"item_id"` column still contains values like `"HOBBIES_1_001"` — we're just calling that column `"part_number"` now.

CODE WALKTHROUGH — DATA QUALITY FLAGS

python

```
def add_data_quality_flags(df):
    # Flag 1: Is demand zero?
    df["is_zero_demand"] = (df["demand_quantity"] == 0).astype(int)
```

TECHNICAL: `(df["demand_quantity"] == 0)` produces a column of True/False values. `.astype(int)` converts True→1, False→0.

BUSINESS: Zero demand is ambiguous:

- It might mean "nobody needed this part today" (genuine zero)
- It might mean "we were out of stock so we couldn't record demand" (censored data — the REAL demand was >0 but we missed it)

Flagging zeros lets analysts and ML models handle them appropriately. Some models treat zeros differently from non-zeros.

```
python
```

```
# Flag 2: Outlier detection
mean_demand = df["demand_quantity"].mean()
std_demand = df["demand_quantity"].std()
df["is_demand_outlier"] = (
    df["demand_quantity"] > mean_demand + 3 * std_demand
).astype(int)
```

TECHNICAL: This is the "3-sigma rule" from statistics. In a normal distribution, 99.7% of values fall within 3 standard deviations of the mean. Anything beyond that is extremely unusual — likely either a data entry error or a genuine extreme event.

BUSINESS: If average daily demand for a part is 10 units and someone enters 9,999 units, that's either:

- A typo (they meant 99 or 9)
- A genuine massive order (factory expansion)

Either way, the ML team needs to know about it. Flagging doesn't DELETE the data — it marks it for investigation.

HOW TO RUN

```
bash
```

```
python src/ingestion/m5_preprocessor.py
```

Expected output:

M5 → AMAT Supply Chain Preprocessing

Loaded raw data:

Sales: 1,440 rows × 8 columns
Prices: 240 rows × 4 columns
Calendar: 30 rows × 11 columns

Mapping M5 columns to AMAT schema...

Mapped 1,440 rows

Adding data quality flags...

✅ Saved to data/processed/m5_amat_mapped.csv

SUMMARY

Total records: 1,440

Unique parts: 8

Unique regions: 3

Demand type breakdown:

NPI	540 (37.5%)
standard	360 (25.0%)
service_campaign	360 (25.0%)
reliability	180 (12.5%)

WHAT TO VERIFY

1. Open `data/processed/m5_amat_mapped.csv` and check that:
 - Column names are AMAT schema names (part_number, not item_id)
 - demand_type values are only: NPI, standard, service_campaign, reliability
 - region values are only: North America, Asia Pacific, Europe
 - No columns from M5 remain (no dept_id, store_id, cat_id)
2. Check the demand type percentages make sense:
 - NPI is highest because 3 out of 8 items map to HOBBIES ($3/8 \text{ items} \times 6 \text{ stores} = 540$)
 - Standard and service_campaign are equal (2 items each)
 - Reliability is lowest (only 1 item)

6. File 4: pipeline.py — The Orchestra Conductor

WHAT THIS FILE DOES

Calls the preprocessor (File 3) and then runs every row through the schema validator (File 2). It separates good records from bad, saves both, and reports statistics.

THE BUSINESS NEED

In production, this pipeline runs automatically every night:

1. New data arrives from factories and suppliers
2. The pipeline cleans, validates, and loads it
3. If the pass rate drops below a threshold, alerts fire
4. Only clean data reaches the ML models

THE CONCEPT: SEPARATION OF CONCERNS

schema.py	→ Knows the RULES	(what valid data looks like)
m5_preprocessor.py	→ Knows the MAPPING	(M5 → AMAT translation)
pipeline.py	→ Knows the PROCESS	(what order to do things)

Each file has ONE job. If validation rules change → edit schema.py only.

If a new data source replaces M5 → write a new preprocessor only.

The pipeline stays the same either way.

CODE WALKTHROUGH — PIPELINE STATISTICS

```
python

class PipelineStats:
    def __init__(self):
        self.start_time = time.time()
        self.total_records = 0
        self.valid_records = 0
        self.rejected_records = 0
        self.null_values_found = 0
```

TECHNICAL: `time.time()` returns the current time as a number (seconds since January 1, 1970). By recording the start time, we can calculate how long the pipeline took.

BUSINESS: These metrics go into monitoring dashboards:

- **Pass rate** → If it drops from 100% to 95%, something changed in the data source. Alert the data engineering team.

- **Processing time** → If it jumps from 2 minutes to 20 minutes, either the data volume grew or something is wrong. Alert ops.
- **Null values** → Track data completeness over time.

python

```
@property
def pass_rate(self):
    if self.total_records == 0:
        return 0.0
    return (self.valid_records / self.total_records) * 100
```

TECHNICAL: `@property` turns a method into something that looks like a field. Instead of calling `stats.pass_rate()`, you just write `stats.pass_rate` — no parentheses. It's syntactic sugar (makes code read more naturally).

CODE WALKTHROUGH — THE MAIN PIPELINE FUNCTION

python

```
def run_pipeline(input_dir="data/raw", output_dir="data/processed", validate=True):
```

TECHNICAL: Default parameters mean you can call `run_pipeline()` with no arguments and it uses sensible defaults. But you CAN override them: `run_pipeline(input_dir="/other/path")`.

python

```
# STEP 1: PREPROCESS
preprocessed_df = run_preprocessing(
    input_dir=input_dir,
    output_path=os.path.join(output_dir, "m5_amat_mapped.csv"),
)
```

TECHNICAL: `os.path.join()` builds file paths correctly for any operating system. On Mac/Linux: "data/processed/file.csv". On Windows: "data\processed\file.csv". Never hardcode slashes.

python

```
# STEP 2: VALIDATE
records_to_validate = []
for _, row in preprocessed_df.iterrows():
    record = {
        "part_number": str(row.get("part_number", "")),
        "demand_quantity": int(row.get("demand_quantity", 0)),
        "npi_signal": str(row["npi_signal"]) if pd.notna(row.get("npi_signal"))
        ...
    }
    records_to_validate.append(record)
```

TECHNICAL: We convert DataFrame rows to dictionaries because our schema validator expects dictionaries. Key details:

1. `str(row.get("part_number", ""))` → Convert to string. `.get()` with a default prevents `KeyError` if the column is missing.
2. `pd.notna(row.get("npi_signal"))` → pandas uses `NaN` for missing values, but our schema expects `None`. `pd.notna()` checks if a value is NOT `NaN/None`. If it IS `NaN`, we pass `None` to the schema.

This `NaN↔None` conversion is a common gotcha. pandas represents missing data as `NaN` (a float), but Python convention is `None`.

python

```
valid_records, errors = validate_batch(records_to_validate)
```

TECHNICAL: This returns TWO lists. Python lets you return multiple values from a function using tuple unpacking: `a, b = function_that_returns_two_things()`

python

```
# STEP 3: SAVE VALID RECORDS
if valid_records:
    valid_df = pd.DataFrame([r.to_dict() for r in valid_records])
```

TECHNICAL: List comprehension: `[r.to_dict() for r in valid_records]`

This is equivalent to:

python

```
list_of_dicts = []
for r in valid_records:
    list_of_dicts.append(r.to_dict())
```

List comprehensions are preferred in Python because they're faster (optimized C loop) and more readable once you're used to them.

HOW TO RUN

```
bash

python -m src.ingestion.pipeline
```

NOTE: We use `python -m src.ingestion.pipeline` (module syntax) instead of `python src/ingestion/pipeline.py` (file syntax) because the pipeline imports from other modules (`from src.ingestion.schema import ...`). Module syntax ensures Python resolves imports correctly from the project root.

Expected output:

 STEP 1: Preprocessing raw data...
(preprocessor output appears here)

 STEP 2: Validating against schema...

 Saved 1,440 valid records to data/processed/m5_amat_validated.csv

PIPELINE RUN REPORT

Total records:	1,440
 Valid:	1,440
 Rejected:	0
Pass rate:	100.0%
Null values found:	1,020
Processing time:	0.14s

UNDERSTANDING THE OUTPUT

1,440 total records: 8 items × 6 stores × 30 days = 1,440

0 rejected: All records passed validation. This is expected with our sample data because `create_sample_data.py` generates clean data. In the real world, you'd see some rejections.

1,020 null values: These are in the `npi_signal` column (which is optional — most days don't have NPI events). This is expected and fine.

0.14s processing time: Very fast because we only have 1,440 rows. With the real 46M row dataset, this would take 2-10 minutes depending on your hardware.

7. File 5: `test_ingestion.py` — The Safety Net

WHAT ARE TESTS AND WHY DO WE WRITE THEM?

Tests are code that checks your other code. You write them **ONCE**, and then every time you change anything, you run the tests to make sure you didn't accidentally break something.

Analogy: Imagine you build a bridge. Before letting cars drive on it, you test it with different weights: a bicycle, a car, a truck, 10 trucks at once. Tests do the same thing for code.

THE 5 TEST GROUPS AND WHAT THEY CHECK

Group 1 — Schema Validation (15 tests) "Does the bouncer work correctly?"

<code>test_valid_record_passes</code>	→ Good passport gets you in
<code>test_empty_part_number_rejected</code>	→ No ID = rejected
<code>test_negative_demand_rejected</code>	→ Can't order -50 parts
<code>test_zero_demand_accepted</code>	→ Zero is valid (nobody needed it)
<code>test_invalid_demand_type_rejected</code>	→ "banana" is not a demand type
<code>test_all_valid_demand_types</code>	→ All 4 real types work
<code>test_invalid_region_rejected</code>	→ "Mars" is not a region
<code>test_all_valid_regions</code>	→ All 3 real regions work
<code>test_negative_unit_cost_rejected</code>	→ Parts can't cost -\$5
<code>test_zero_unit_cost_rejected</code>	→ Parts can't be free (\$0)
<code>test_invalid_date_rejected</code>	→ "2015-13-45" isn't a date
<code>test_valid_date_accepted</code>	→ "2023-12-31" works
<code>test_null_npi_signal_accepted</code>	→ NPI signal is optional (None ok)
<code>test_to_dict_has_all_fields</code>	→ Conversion to dict works
<code>test_empty_description_rejected</code>	→ Description can't be blank

Group 2 — Batch Validation (5 tests) "Does the assembly line sort correctly?"

```
test_all_valid_records_pass      → 10 good → 10 in, 0 rejected
test_mixed_batch_separated      → 3 good + 2 bad → 3 in, 2 rejected
test_errors_include_row_number  → Errors say WHICH row failed
test_empty_batch                → 0 records → 0 in, 0 rejected
test_all_invalid_records_rejected → 3 bad → 0 in, 3 rejected
```

Group 3 — M5 Preprocessing (6 tests) "Does the translator work?"

```
test_all_departments_have_mapping → Every dept maps to something
test_all_stores_have_mapping      → Every store maps to a region
test_mapped_demand_types_valid   → Mapped values match schema
test_mapped_regions_valid        → Mapped regions match schema
test_map_columns_output_structure → Old M5 columns are gone
test_specific_mapping_values      → FOODS_3/WI_3 → reliability/Europe
```

Group 4 — Data Quality Flags (5 tests) "Do the warning labels work?"

```
test_zero_demand_flagged        → Zero demand gets flag = 1
test_nonzero_demand_not_flagged → Normal demand gets flag = 0
test_npi_signal_detected        → Events detected correctly
test_outlier_detected           → 999 among 10s flagged as outlier
test_quality_columns_added       → All 3 flag columns exist
```

Group 5 — End-to-End Pipeline (5 tests) "Does the whole car drive?"

```
test_pipeline_creates_output    → Output CSV exists and isn't empty
test_no_nulls_in_required_fields → Required fields are complete
test_output_demand_types_valid  → Only valid types in output
test_output_regions_valid       → Only valid regions in output
test_no_negative_demand_in_output → No negative quantities anywhere
```

HOW TO RUN TESTS

```
bash

# Run with our custom runner (shows / for each test):
python tests/test_ingestion.py

# OR run with pytest (industry standard):
python -m pytest tests/test_ingestion.py -v
```

Expected output:

Schema Validation

- ✅ All 4 demand types accepted
- ✅ Empty part_number correctly rejected
- ✅ Negative demand correctly rejected
- ... (15 tests)

Batch Validation

... (5 tests)

M5 Preprocessing

... (6 tests)

Data Quality Flags

... (5 tests)

End-to-End Pipeline

... (5 tests)

TEST RESULTS

Total: 36

✅ Passed: 36

❌ Failed: 0

🎉 ALL TESTS PASSED!

8. How to Execute Everything (Step by Step)

THE COMPLETE SEQUENCE

bash


```

# Step 1: Navigate to project folder
cd agentic-supply-chain-orchestrator

# Step 2: Activate virtual environment
source venv/bin/activate          # Mac/Linux
# venv\Scripts\activate          # Windows

# Step 3: Generate sample data
python data/raw/create_sample_data.py

# Step 4: Run the preprocessor (standalone test)
python src/ingestion/m5_preprocessor.py

# Step 5: Run schema validation (standalone test)
python src/ingestion/schema.py

# Step 6: Run the full pipeline
python -m src.ingestion.pipeline

# Step 7: Run ALL tests
python tests/test_ingestion.py

# Step 8: Verify output files
ls -la data/processed/
cat data/processed/m5_amat_validated.csv | head -5

```

WHAT TO CHECK AFTER EACH STEP

```

Step 3 → 3 CSV files exist in data/raw/
Step 4 → m5_amat_mapped.csv exists in data/processed/
Step 5 → "Schema validation working correctly!" printed
Step 6 → "Pipeline complete! 1,440 records ready for ML models." printed
Step 7 → "ALL TESTS PASSED!" printed (36/36)
Step 8 → m5_amat_validated.csv has columns: part_number, demand_type, etc.

```

9. Metrics & What They Mean

PHASE 1 METRICS (for your resume and interviews)

METRIC	VALUE	WHAT IT MEANS
--------	-------	---------------

Schema validation pass rate	100%	Every record is clean
Records processed	1,440	(46M+ with real data)
Rejected records	0	No bad data got through
Null values in required fields	0	All critical fields filled
Data quality flags added	3	Zero-demand, outlier, NPI
Test count	36	Comprehensive coverage
Tests passing	36/36	100% test pass rate
Processing time	0.14s	Fast enough for production
Test groups	5	Unit, integration, E2E

HOW TO TALK ABOUT THESE IN INTERVIEWS

Don't just say "100% pass rate." Explain WHY it matters:

"I achieved 100% validation pass rate, which means zero data corruption reaching the ML pipeline. This eliminates an entire class of bugs where malformed data could silently produce incorrect forecasts. Each record is validated against 7 business rules covering type safety, range constraints, and referential integrity."

10. Interview Preparation

QUESTION 1: "Walk me through your data pipeline."

YOUR ANSWER:

"My ETL pipeline has three stages with clear separation of concerns. Extraction reads three raw CSV files — sales, pricing, and calendar data from the M5 Forecasting dataset. Transformation maps the retail schema to a semiconductor supply chain schema — renaming columns, mapping departments to demand types, and stores to geographic regions. The mappings preserve statistical properties: HOBBIES maps to NPI because both exhibit sudden spikes followed by gradual decline. Loading writes validated records to processed CSVs and logs rejected records with row numbers for debugging. The pipeline uses error accumulation rather than fail-fast, achieving 100% pass rate across all records."

QUESTION 2: "Why M5 and not real supply chain data?"

YOUR ANSWER:

"Real supply chain data is proprietary. M5 was specifically chosen because it has 46 million real-world records with genuine seasonality, event-driven demand spikes, and hierarchical structure — items within

departments within stores maps naturally to parts within categories within regions. M5 is the academic gold standard for forecasting research, used in published papers and Kaggle competitions. The statistical patterns are preserved through domain mapping, so our ML models train on realistic demand behavior."

QUESTION 3: "How do you handle bad data?"

YOUR ANSWER:

"Every record passes through 7 validation checks: type checking for all fields, range validation for quantities and prices, allowed-value checking against controlled vocabularies for demand types and regions, date format validation, and required-field enforcement. I use error accumulation rather than fail-fast, so one bad record doesn't block 99,999 good ones. Rejected records are logged with row numbers and specific error messages for debugging. In production, error rates would feed into a monitoring dashboard with alerting thresholds."

QUESTION 4: "What would you improve?"

YOUR ANSWER:

"Three things. First, chunked processing — reading the CSV in batches of 100K rows instead of loading 46M rows into memory at once. Second, I'd use Parquet instead of CSV for the processed output — it's columnar storage that's 5-10x faster to read and 3-5x smaller on disk. Third, I'd add Great Expectations or Pandera for production-grade data profiling with statistical drift detection — alerting when the distribution of demand changes significantly between batches."

QUESTION 5: "Explain separation of concerns in your project."

YOUR ANSWER:

"Schema validation, data transformation, and pipeline orchestration are in separate files with no circular dependencies. If validation rules change, only schema.py is modified. If we switch from M5 to a different data source, only the preprocessor changes — the schema and pipeline remain untouched. Each component is independently testable: I have 15 tests for schema validation alone, 6 for preprocessing, and 5 for end-to-end pipeline verification. This modularity also enables team parallelism — one engineer can work on the schema while another works on the preprocessor."

What's Next: Phase 2

Phase 1 gives you CLEAN DATA. Phase 2 uses that clean data to PREDICT future demand using two ML models:

1. XGBoost — finds complex patterns across many features
2. ARIMA — captures seasonality and trends over time

The clean `m5_amat_validated.csv` we produced is the INPUT for Phase 2.

When you're ready, we'll build the forecasting models together.